



Unit IV: PL/SQL Programming

4.1

- Introduction of PL/SQL
- Advantages of PL/SQL
- The PL/SQL Block Structure
- PL/SQL execution environment
- PL/SQL Data Types, Variables, Constants

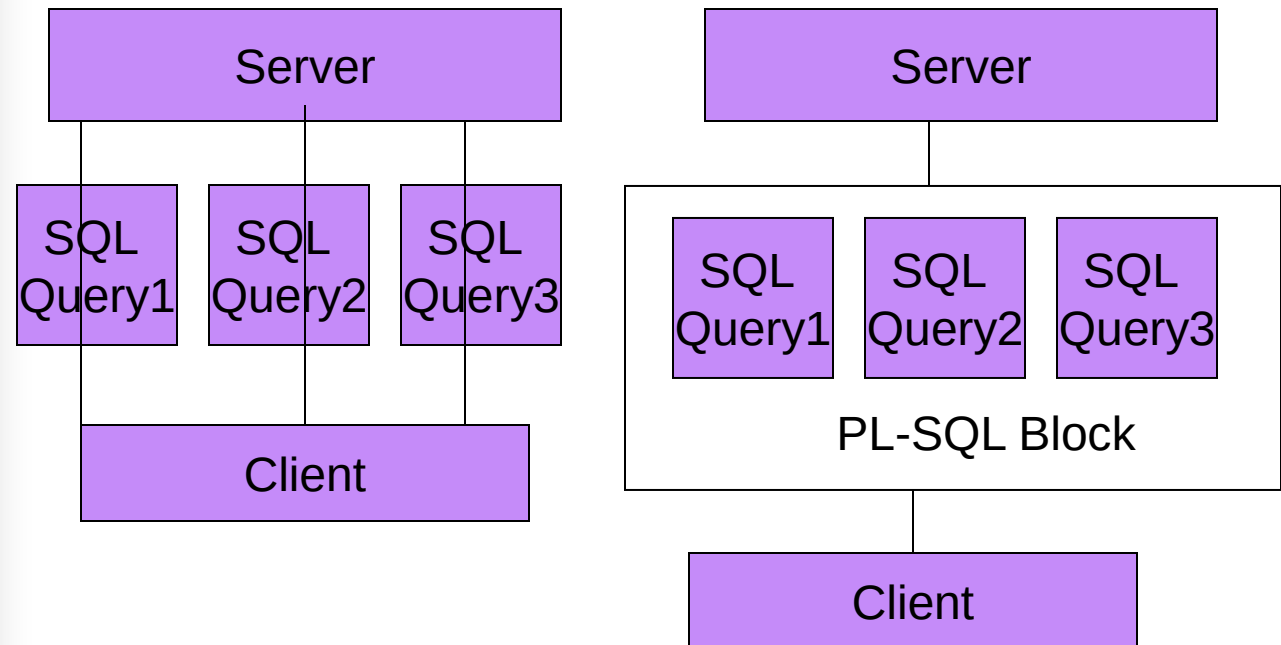


Introduction of PL SQL

- PL/SQL stands for Procedural Language/SQL.
- PL/SQL extends SQL by adding constructs found in procedural languages like procedures, loops, variables, objects etc.
- Resulting a structural language that is more powerful than SQL



Advantage of PL SQL



- In case of SQL to send 3 queries we will need three network trips between client and server.
- In PL-SQL we bundle any number of queries in a block and in single network trip task is done.



Language features

- A procedural language, which provides the functionality of decision making, iteration and many more features of procedural programming languages.
- Execute a number of queries in one block using single command.
- Supports constructs like any other 4th generation language such as:
 - Variables and Data types
 - Loops and Control statements
 - Procedures and Functions
 - Packages
 - Triggers
 - Objects
 - Records (Its like structure in C language)
- These are stored in the database for reuse by applications.
- Provides a feature to handle the exception which occurs in PL/SQL block known as exception handling block.
- Applications written in PL/SQL are portable to computer H/W or operating system where Oracle is operational.
- PL/SQL Offers extensive error checking.



Differences between SQL and PL/SQL:

SQL	PL/SQL
SQL is a single query that is used to perform DML and DDL operations.	PL/SQL is a block of codes that used to write the entire program blocks/ procedure/ function, etc.
It is declarative, that defines what needs to be done, rather than how things need to be done.	PL/SQL is procedural that defines how the things needs to be done.
Execute as a single statement.	Execute as a whole block.
Mainly used to manipulate data.	Mainly used to create an application.
Cannot contain PL/SQL code in it.	It is an extension of SQL, so it can contain SQL inside it.



The PL/SQL Block Structure

Declare

Declaration statements;
<All Variables, cursors, exception etc
are declared here>

Begin

Executable statements;
<All programming logic , queries ,
program statements are written here>

Exception

Exception handling statements;
<All Error Handling code is written here>

End;

--It ends the program



PL SQL nested block

<<Outer Block>>

Declare

Begin

<<Inner Block>>

Declare

Begin

Exception

End;

Exception

End;

PL SQL Block



Remember :

Declare is optional and only required when variables need to be declared.

Exception is optional and required when Error/Exception handling is done.

Begin and End are mandatory as all logic and queries are written inside it.

Declare	----- optional
Begin	----- mandatory
Exception	----- optional
End;	----- mandatory

PL SQL program- Sample I



BEGIN

Insert into Dept values(70,'HR','Pune');

Insert into Dept values(80,'WELFARE','Mumbai');

Insert into Dept values(90,'SUPPORT','Pune');

END;

--This program will insert three records at the same time in the table dept.

PL SQL program- Sample II

-- This program displays the sum of two numbers

DECLARE

num1 Number;

num2 Number;

total Number;

} Variable Declaration

BEGIN

num1:= 20;

num2:= 40;

total:=num1+num2; } Value Assignment
DBMS_OUTPUT.PUT_LINE('SUM OF TWO Nos= ' || total);

END;

/

```
SQL> @ c:\notes1\psum.sql
```

```
SUM OF TWO Nos= 60
```

```
PL/SQL procedure successfully completed.
```

```
SQL>
```



PL SQL program- Sample III



-- This program displays the sum of two numbers

DECLARE

num1 Number(5):= 20;

num2 Number(5):= 40;

total Number(6);

BEGIN

total:=num1+num2; ← Assigning sum of num1 & num2 to total
DBMS_OUTPUT.PUT_LINE('SUM OF TWO Nos= ' || total);

END;

/

```
SQL> @ c:\notes1\psum.sql
```

```
SUM OF TWO Nos= 60
```

```
PL/SQL procedure successfully completed.
```

```
SQL>
```

PL SQL program- Sample IV

-- This program displays the sum of two numbers

DECLARE

num1 Number;
num2 Number;
total Number;

} Variable Declaration

BEGIN

num1:=&Number1;
num2:=&Number2;

} Interactive Value Assignment

total:=num1+num2; ← Assigning sum of num1 & num2 to total
DBMS_OUTPUT.PUT_LINE('SUM OF TWO Nos= ' || total);

END;

/

```
SQL> ed c:\notes1\psum.sql
```

```
SQL> @ c:\notes1\psum.sql
```

```
Enter value for number1: 20
```

```
old 6:      num1:=&Number1;
```

```
new 6:      num1:=20;
```

```
Enter value for number2: 40
```

```
old 7:      num2:=&Number2;
```

```
new 7:      num2:=40;
```

```
SUM OF TWO Nos= 60
```

```
PL/SQL procedure successfully completed.
```

```
SQL>
```



PL SQL program- Sample V

-- This program displays the sum of two numbers

DECLARE

num1 Number:= &Number1;

num2 Number:= &Number2;

total Number;

} Variable Declaration &
Value Assignment

BEGIN

total:=num1+num2; ← Assigning sum of num1 & num2 to total
DBMS_OUTPUT.PUT_LINE('SUM OF TWO Nos= ' || total);

END;

/

```
SQL> ed c:\notes1\psum.sql
```

```
SQL> @ c:\notes1\psum.sql
```

```
Enter value for number1: 20
```

```
old 6: num1:=&Number1;
```

```
new 6: num1:=20;
```

```
Enter value for number2: 40
```

```
old 7: num2:=&Number2;
```

```
new 7: num2:=40;
```

```
SUM OF TWO Nos= 60
```

```
PL/SQL procedure successfully completed.
```

```
SQL>
```





PL SQL program- Sample VI

-- This program displays the tables of a user

BEGIN

dbms_output.put_line (user || ' Tables in the database:');

FOR t IN (SELECT table_name FROM user_tables)

LOOP

dbms_output.put_line(t.table_name);

END LOOP;

END;

/

```
SQL> BEGIN
  2  dbms_output.put_line (user || ' Tables in the database:');
  3  FOR t IN (SELECT table_name FROM user_tables)
  4  LOOP
  5      dbms_output.put_line(t.table_name);
  6  END LOOP;
  7  END;
  8  /
```

SCOTT Tables in the database:

T2

FEE_DETAIL

STUDENT

QUALIFICATION

STUDENTS

GRADES

ENROLMENT

CONTACT1

EMPLOYEE

T1

SALGRADE

BONUS

EMP

DEPT

PL/SQL procedure successfully completed.

SQL> _



PL/SQL execution environment



Save, Edit and Execute program

- Type program in SQL * plus
- To save : **Save <File Name>**
Program is saved in the bin directory to save in other folder give complete path.
e.g.: **Save 'C:\NOTES1\test.sql'**
- To make changes:
Edit <File Name>
To edit program saved in folder other than bin
Edit 'C:\NOTES1\test.sql'
- To Execute:
@ File Name
To execute program saved in folder other than bin.
@ 'C:\NOTES1\test.sql'


```
SQL> BEGIN
  2      dbms_output.put_line (user || ' Tables in the database:');
  3      FOR t IN (SELECT table_name FROM user_tables)
  4      LOOP
  5          dbms_output.put_line(t.table_name);
  6      END LOOP;
  7  END;
  8  /
```

SCOTT Tables in the database:

T2
FEE_DETAIL
STUDENT
QUALIFICATION
STUDENTS
GRADES
ENROLMENT
CONTACT1
EMPLOYEE
T1
SALGRADE
BONUS
EMP
DEPT

PL/SQL procedure successfully completed.

```
SQL> save c:\notes1\test.sql
Created file c:\notes1\test.sql
SQL> ed c:\notes1\test.sql
```

```
SQL> @ c:\notes1\test.sql
SCOTT Tables in the database:
```

T2
FEE_DETAIL
STUDENT
QUALIFICATION
STUDENTS
GRADES
ENROLMENT
CONTACT1
EMPLOYEE
T1
SALGRADE
BONUS
EMP
DEPT

PL/SQL procedure successfully completed.



Important Keywords

Following are the keywords in PL-SQL , should not be used as a variable name.



Keywords

- DECLARE
- BEGIN
- END
- EXCEPTION
- LOOP , END LOOP
- IF , ELSE , ELSIF , END IF
- CURSOR
- PROCEDURE
- FUNCTION
- PACKAGE
- TRIGGER



Keywords

- GRANT
 - REVOKE
 - FOR
 - WHILE
 - CASE
 - VARRAY
 - TYPE
 - OBJECT
- 



Operators

■ Important operators in PL SQL

- Airthmetic : $(+, -, *, /)$
- Logical: (AND, OR, NOT)
- Comparison: $(<=, <, >, =)$
- Comments (Two hyphens): $--$
- Assignment operator:

In PL SQL assignment operator is $:=$

So to assign values we need to write $:=$

Examples:

$z := x + y$

$z := x$

$z := 100$

$name := 'MHSSP'$



Operators

Important operators in PL SQL

- Line ends with operator: ;
- To join two strings: ||
- To accept value: &
- Power **
2**3 means 2 raise to power 3
- In loop we use for giving range ..
Example:
For X in 1..5 means 1 to 5
- Non numeric data (string or date)
is written in single quote: ' '



Accept a value

■ Example 1:

```
num1 := &Number1;
```

At run time this will prompt as Follows

Enter a value for Number1:

Whatever value user will enter here will be assign to variable num1

■ Example 2:

```
name := '&Name';
```

At run time this will prompt as Follows

Enter a value for Name:

Whatever value user will enter here will be assign to variable name

‘ ’ is used in case if entered data is not numeric



Display value

- To display on same line:
`dbms_output.put()`
- To display on new line.
`dbms_output.put_line()`
- Here **dbms_output** is a **Oracle package** its like **header file** or **library** in **C language**.
- .Put and .Put_Line are functions like printf in 'C' language



Display value on Output Screen

- On SQL prompt after Login we need to set one command to see displayed values.

SET SERVEROUTPUT ON



Display value : Examples

```
Dbms_output.put ('Mohammad ');
```

```
Dbms_output.put ('Haji ');
```

```
Dbms_output.put ('Saboo ');
```

```
Dbms_output.put ('Siddik');
```

```
Dbms_output.put_line(' ');
```

It will display **Mohammad Haji Saboo Siddik** on same line.

```
SQL> begin
  2  Dbms_output.put ('Mohammad ');
  3  Dbms_output.put ('Haji ');
  4  Dbms_output.put ('Saboo ');
  5  Dbms_output.put ('Siddik');
  6  Dbms_output.put_line(' ');
  7  end;
  8  /
Mohammad Haji Saboo Siddik

PL/SQL procedure successfully completed.
```

Note :

It is important that at least once you write **.put_line** after any number of **.put** functions else values are not displayed.

```
SQL> begin
  2  Dbms_output.put ('Mohammad ');
  3  Dbms_output.put ('Haji ');
  4  Dbms_output.put ('Saboo ');
  5  Dbms_output.put ('Siddik');
  6  end;
  7  /

PL/SQL procedure successfully completed.
```




Display value : Examples



```
Dbms_output.put_line ('Mohammad');  
Dbms_output.put_line ('Haji');  
Dbms_output.put_line ('Saboo');  
Dbms_output.put_line ('Siddik');
```

It will display



Mohammad
Haji
Saboo
Siddik

on different lines.



```
SQL> BEGIN  
2   Dbms_output.put_line ('Mohammad');  
3   Dbms_output.put_line ('Haji');  
4   Dbms_output.put_line ('Saboo');  
5   Dbms_output.put_line ('Siddik');  
6   END;  
7   /  
Mohammad  
Haji  
Saboo  
Siddik  
  
PL/SQL procedure successfully completed.
```



DML operations in PL/SQL

- All DML operations (**Insert/Update/Delete/Select**) are to be written in **Begin** part of the block.
- No change in the Syntax of Insert, Update and Delete, it is same as SQL.
- **Select** syntax is different than SQL, it contains **INTO** clause.
- If Select query can return more than one rows then we should always use cursors .



Select Syntax for a Single Row Query.

- Select column1, column2
INTO Variable1,Variable2
From Table Name
Where <condition>
- The only change is as many columns we want to get from the query we need to declare that many variables and use INTO clause.
- All other parts of query are unchanged
- If Where condition here is such that query will return multiple records then CURSOR should be used. Without that it will give error.



Data Types in PL SQL

■ Scalar Types

- **Char**

CHAR datatype to store fixed-length character data.
Maximum size = 2000 bytes

- **Varchar2**

VARCHAR2 datatype to store variable-length character . Maximum size = 4000 bytes

- **Number**

Number types store numeric data (integers, real numbers, and floating-point numbers), represent quantities, and do calculations.

- **Binary_Integer**

The BINARY_INTEGER datatype to store signed integers

(-2^{31} to 2^{31})

- **Date**

DATE datatype to store fixed-length datetimes



Data Types in PL SQL

■ Scalar Types

- **Long**

The LONG datatype to store variable-length character strings. The LONG datatype is like the VARCHAR2 datatype, except that the maximum size of a LONG value is 32760 bytes.

- **NChar**

To store multi byte fixed length character data. Its same as Char only difference is it is used to store characters of different language like Japanese, Chinese etc.

Number of characters it can store depend on language.

- **NVarchar**

To store multi byte variable length character data. Its same as Varchar2 only difference is it is used to store characters of different language like Japanese , Chinese etc.

Number of characters it can store depend on language.



Data Types in PL SQL

■ Composite Types

- **Record**

Its like structure in C Language.

- **Table**

Its like Array in C Language.

This Array type is un-constrained array

- **VArray**

Its like Array in C Language.

This Array type is constrained array

■ Reference Types

- **Ref Cursor**

Its used for dynamic cursor.



Data Types in PL SQL

■ **LOB**

● **BLOB**

Binary Large Object A column or variable of type BLOB can store up to 4GB of binary data in each record.

● **CLOB**

Character Large Object A column or variable of type CLOB can store up to 4GB of character data in each record.

● **BFILE**

It can store a file of size 4GB externally outside database for each record and can refer to that from inside the database.

- Column or variable of this type can be accessed only using a Oracle package DBMS_LOB.
- This should be used only if required to store a large amount of data in each record of a table
- We should avoid making un-necessary use of LOB's



Variable Declaration in PL SQL

- Variables are always declared in **DECLARE** section of the program.

variable_name <Data Type>

- Various way to declare them

v_empno number;

v_ename varchar2;

v_job char(10);



Variable Declaration in PL SQL

- Dynamic and preferred way to declare a variable, when working with table

variable_name **TableName.ColName%Type**

v_empno **Emp.Empno%Type**;

v_ename **Emp.Ename%Type**;

v_deptno **Dept.Deptno%Type**;

- Advantages of declaring in above way.
 - Variable will always have same datatype as column
 - Any change in column will change the type of variable also, so we need not have to change and recompile the program to run.



Variable Declaration in PL SQL

■ %RowType

variable_name **TableName%RowType**

v_emp **Emp%RowType**;

■ Advantages of declaring in above way.

- Variable will become like a structure variable in C (i.e. **v_emp** will have same structure like **Emp Table**) and we can refer to individual element as follows:

v_emp.empno

v_emp.ename

v_emp.sal



Variable Declaration in PL SQL

- **Type** : We can also make our own type in program and use in the declare section to declare variable.

Type t_name is Varchar2(20);

-- now we can make variable of this type

name **t_name**;

name2 **t_name**;

job **t_name**;

name, name2 and job will become varchar2(20)



Thank you !!